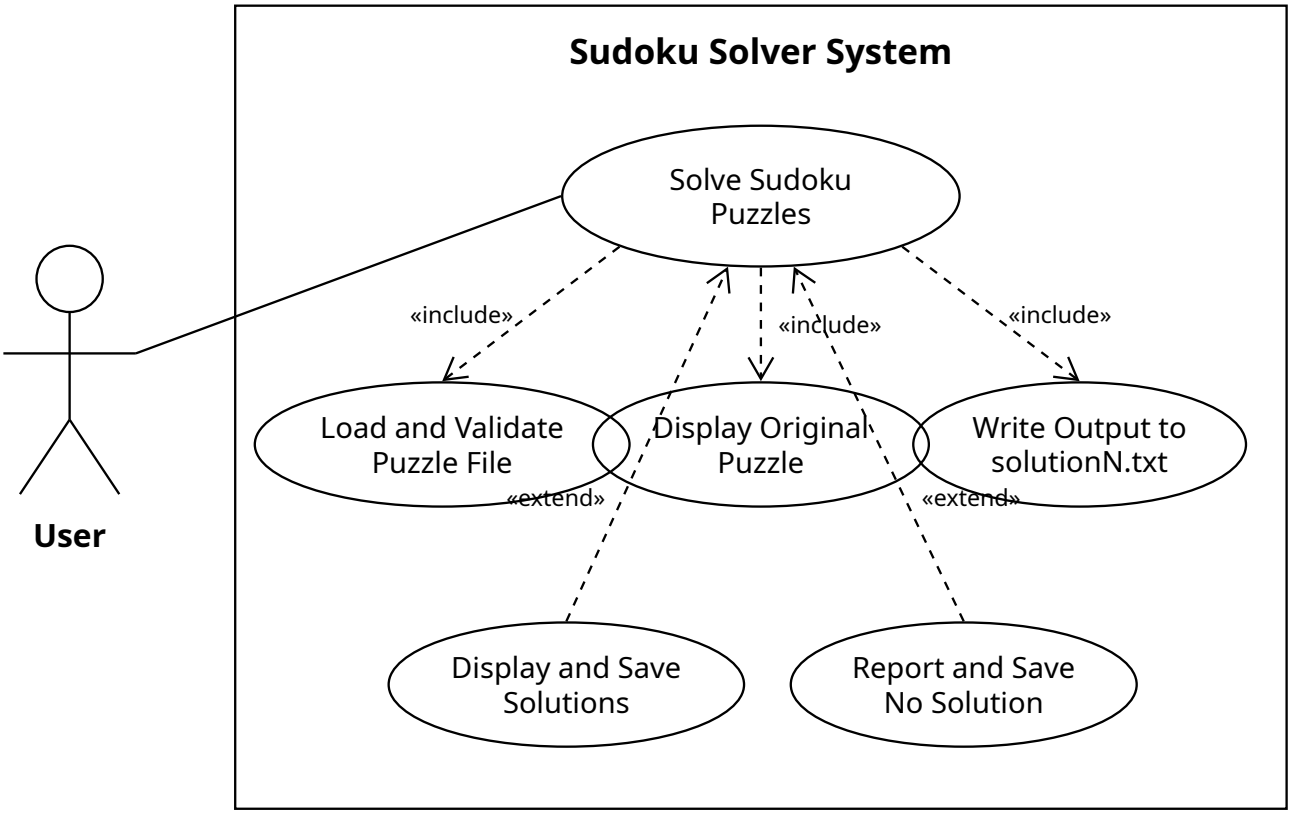
UML Use Case Diagram - Sudoku Solver



The User starts the Sudoku Solver. The system loads and validates each puzzle file (`puzzle1.txt` to `puzzle5.txt`), opens a corresponding output file (`solution1.txt` to `solution5.txt`), displays the original grid, and both displays to the console and writes to `solutionN.txt` all valid solutions (followed by `Solutions found: n`) or reports `No solution found.`. The five files are processed automatically.

Use Case Specification

a. Identify / Discover a Use Case

Use case: Solve Sudoku Puzzles
Primary actor: User
System: Sudoku Solver
Goal: Process five Sudoku puzzle files (`puzzle1.txt` to `puzzle5.txt`), display every valid solution to the console, and write all formatted solutions (or a no-solution message) to corresponding `solution1.txt` to `solution5.txt` files.

b. Brief Description

The User runs the Sudoku Solver program. The system reads five puzzle files (`puzzle1.txt` to `puzzle5.txt`), checks their contents, opens a corresponding output file (`solution1.txt` to `solution5.txt`), displays each original puzzle, and searches for all valid solutions. It prints each solution to the console and writes every valid solution (formatted identically to the input puzzle files) followed by `Solutions found: n` to `solutionN.txt`, or outputs `No solution found.` when no solution exists.

c. Step-by-Step Flow (Outline)

1. User starts the Sudoku Solver program.
2. System opens the next puzzle file, from `puzzle1.txt` through `puzzle5.txt`.
3. System reads the 9×9 puzzle and checks the file contents.
4. System opens the corresponding output file, from `solution1.txt` through `solution5.txt`.
5. System displays the original puzzle to the console.
6. System checks the starting numbers for conflicts.
7. System fills empty cells using recursion and backtracking.
8. System displays every solution to the console and writes each solution grid in the input format to `solutionN.txt`, followed by `Solutions found: n` (or writes `No solution found.` if none exist).
9. System closes `solutionN.txt`, repeats for the remaining files, and then finishes.

d. Detailed Description

Use case name	Solve Sudoku Puzzles
Primary actor	User
Trigger	User starts the program
Preconditions	Program can run; `puzzle1.txt` to `puzzle5.txt` are expected in the working folder; working folder has write permissions for `solution1.txt` to `solution5.txt`
Success result	Original puzzle and all valid solutions are displayed on the console; all solutions (in input file format) followed by "`Solutions found: n`" are written to `solutionN.txt`
Other results	"`No solution found.`" is displayed and written to `solutionN.txt`, or an error message is shown for an unavailable or invalid file

Detailed Use Case: Events and Alternatives

Basic Flow of Events

10. **Start program:** The User runs the Sudoku Solver. The system initializes the input file list (`puzzle1.txt` to `puzzle5.txt`) and output file list (`solution1.txt` to `solution5.txt`), beginning with `puzzle1.txt` and `solution1.txt`.
11. **Open and read file:** The system opens the current puzzle file and reads 81 cells, each containing a digit from 1 to 9 or an underscore (_).
12. **Open output file:** The system opens the corresponding `solutionN.txt` file for writing.
13. **Display puzzle:** The system displays the puzzle file name and original 9×9 grid to the console.
14. **Check starting puzzle:** The system checks that the given digits do not repeat in any row, column, or 3×3 block.
15. **Search for solutions:** The system tries valid digits in empty cells using recursion.
16. **Backtrack when needed:** If a choice cannot complete the puzzle, the system removes it and tries another digit.
17. **Show and write results:** For each complete solution found, the system prints the board to the console and writes the 9×9 solution grid (in the exact input format) to `solutionN.txt`. Once the search finishes, the system writes a single summary line "`Solutions found: n`" (or "`No solution found.`") to `solutionN.txt` and closes the output file.
18. **Continue and finish:** The system repeats the process for the remaining files and ends after the fifth file (`puzzle5.txt` / `solution5.txt`).

Alternative Flows

A1. Input file cannot open: Show an error on the console for that puzzle file and continue with the next puzzle without generating `solutionN.txt`.

A2. Invalid file contents: Show an error on the console for missing, extra, or invalid characters and continue with the next puzzle.

A3. Output file cannot open: Show an error on the console indicating `solutionN.txt` could not be opened and continue with the next puzzle.

A4. Conflicting starting digits: Display "`No solution found.`" on the console, write a single line "`No solution found.`" to `solutionN.txt`, close the file, and continue without attempting to solve it.

A5. No valid solution: Display "`No solution found.`" on the console, write a single line "`No solution found.`" to `solutionN.txt`, close the file, and continue to the next puzzle.

A6. Multiple solutions: Print every valid solution to the console and write every valid solution grid to `solutionN.txt` followed by "`Solutions found: n`" on the final line.